

DevOps e Site Reliability Engineering

Processo de Software

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software





Site Reliability Engineering (SRE)

novatec

Editado por Betsy Beyer, Chris Jones,
Jennifer Petoff e Niall Richard Murphy

Livro Base da Aula

Site Reliability Engineering

How Google Runs Production Systems

- **Autores:** Betsy Beyer, Chris Jones, Jennifer Petoff & Niall Richard Murphy
- **Editora:** O'Reilly Media / Google SRE
- **Foco:** Engenharia de Confiabilidade de Sistemas, automação, cultura sem culpa e gerenciamento de incidentes em escala global.

1 Abordagem Tradicional com Sysadmins

- Conceitos Preliminares
- Contexto Histórico e Processo
- O Papel do Sysadmin e Escalabilidade

2 Vantagens e Desvantagens do Modelo Tradicional

- Vantagens do Modelo Sysadmin
- Desvantagens e Custos Ocultos

3 O Conflito Dev vs Ops

- Origens e Dinâmica do Conflito

4 A Abordagem do Google: SRE

- Definição e Filosofia
- Composição e Habilidades da Equipe
- Cenários de Atuação SRE
- Origem e Comparativo

5 Práticas e Regras Operacionais da SRE

- Perfil e Mindset do SRE
- Regra dos 50% e Mecanismos de Controle

6 Benefícios e Desafios do Modelo SRE

- Vantagens Estratégicas
- Desafios e Barreiras de Adoção

7 DevOps vs SRE e Princípios Fundamentais

- DevOps vs SRE
- Responsabilidades e Válvula de Segurança

8 Plantão, Postmortem e Error Budget

- Postmortem e Cultura Blameless
- Error Budget e Disponibilidade

Conceito Fundamental: O que é um Pager?

Definição e Origem

Historicamente, o **pager** (ou bip) era um pequeno dispositivo físico de bolso que emitia um sinal sonoro estridente quando uma emergência ocorria. Hoje, em SRE, o termo refere-se ao **sistema de alerta de emergência crítica**.



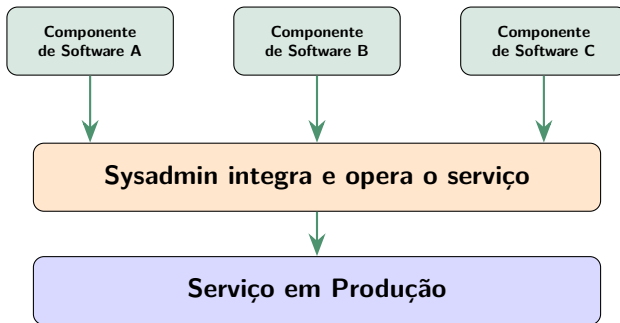
Aplicação no Dia a Dia

Estar “on-call” (de plantão) significa carregar o pager: se o sistema cair às 3h da manhã, o pager toca e o engenheiro deve acordar para resolver o incidente.



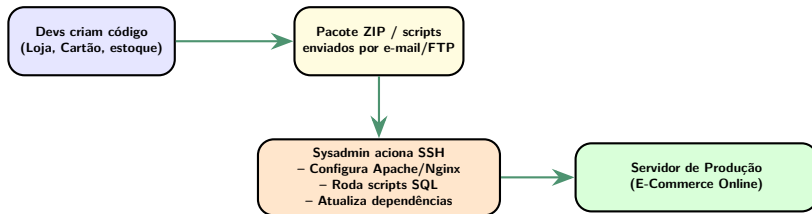
Como as empresas operavam seus sistemas?

Historicamente, as empresas empregaram **administradores de sistemas** (sysadmins) para operar sistemas computacionais complexos.



Cenário Real: Lançamento de Nova Versão da Loja Virtual

Como o fluxo tradicional funcionava na prática para uma aplicação web com banco de dados?



Fluxo Manual

O Desenvolvedor entrega o pacote e o **Sysadmin assume total responsabilidade** por fazer o código funcionar no servidor de produção.

O que acontece em datas críticas (ex: Black Friday)?

A dependência de intervenção humana em processos manuais gera falhas severas.

Erro Humano em Scripts

Execução manual de SQL ou configs pode derrubar a loja em produção.

Servidores "Snowflake"

Apenas o Sysadmin conhece o estado exato da máquina.

Demora no Rollback

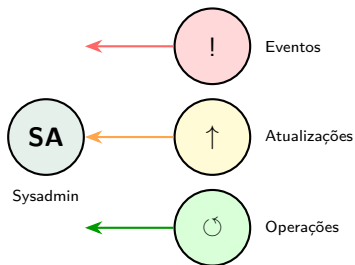
Se der erro, desfazer a atualização manual pode levar horas.

Gargalo Operacional

Acúmulo de tarefas de deploy paralisa novas funcionalidades.

Responsabilidades

- Operar o serviço no dia a dia
- Responder a **eventos** (incidentes)
- Aplicar **atualizações** e mudanças
- Manter a infraestrutura estável

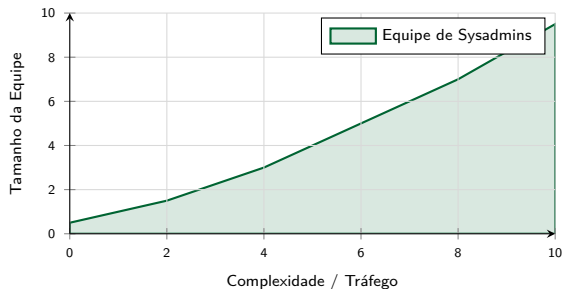


Escalabilidade do Modelo Tradicional

Problema Central

À medida que a **complexidade** e o **volume de tráfego** aumentam, a equipe de sysadmins precisa crescer **proporcionalmente**.

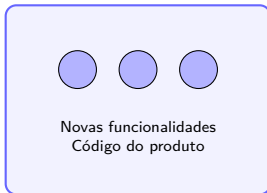
Custo cresce linearmente!



Divisão Natural

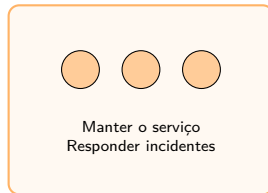
Como sysadmins e desenvolvedores têm **habilidades distintas**, são separados em equipes: **Desenvolvimento** e **Operações**.

Desenvolvimento



-SEPARAÇÃO

Operações (Ops)



Vantagens do Modelo Tradicional

1. **Fácil de implementar** — paradigma conhecido
2. **Muitos exemplos** disponíveis no mercado
3. **Talentos disponíveis** — profissionais prontos
4. **Ferramentas prontas** — softwares COTS e integradores
5. **Sem reinventar a roda** — infraestrutura existente

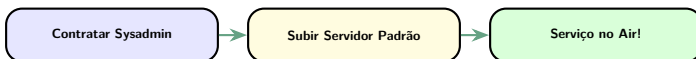
Vantagem 1: Facilidade de Implementação

Explicação Conceitual

Por ser um paradigma estabelecido há décadas, a empresa não precisa reinventar papéis organizacionais. As tarefas de Ops são claras: montar servidores, instalar software e responder a incidentes.

Estudo de Caso: Startup de Software SaaS (FinTechX)

Uma startup de gestão financeira contrata **1 Sysadmin sênior**. Em poucas semanas, ele configura servidores Linux (Nginx + MySQL) em um provedor de nuvem tradicional. A empresa começa a operar rapidamente sem gastar meses desenhando processos complexos.



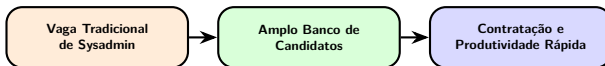
Vantagem 2: Abundância de Talentos no Mercado

Explicação Conceitual

O mercado de trabalho possui um vasto grupo de profissionais formados com certificações e conhecimentos consolidados (Linux LPIC, Windows Server, Redes Cisco).

Estudo de Caso: Empresa de Software RH (RHSoft)

A *RHSoft* precisa substituir um membro da equipe de operações. Ela abre uma vaga de "Administrador de Redes e Sistemas". Em poucos dias, recebe dezenas de currículos de profissionais já capacitados a operar seus servidores Apache e PostgreSQL, sem necessidade de treinamento intensivo.



Vantagem 3: Ferramentas Prontas e Integradores

Explicação Conceitual

Existe um ecossistema maduro de softwares comerciais e open-source (Zabbix, Nagios, cPanel, VMware), além de consultorias terceirizadas de TI prontas para ajudar.

Estudo de Caso: ERP Médico (MedApp)

A empresa desenvolvedora do *MedApp* prefere focar 100% do seu time de dev na criação de funcionalidades médicas. Em vez de desenvolver um sistema próprio de monitoramento, ela instala o **Zabbix** e contrata uma consultoria de TI local para gerenciar os backups diários.

Zabbix / Nagios
(Monitoramento)

VMware / cPanel
(Gestão)

Consultorias TI
(Suporte Terceirizado)

Vantagem 4: Sem Reinventar a Roda

Explicação Conceitual

Não há necessidade de desenhar arquiteturas de automação sob medida do zero. A equipe iniciante utiliza scripts padrão e guias já testados por milhares de empresas.

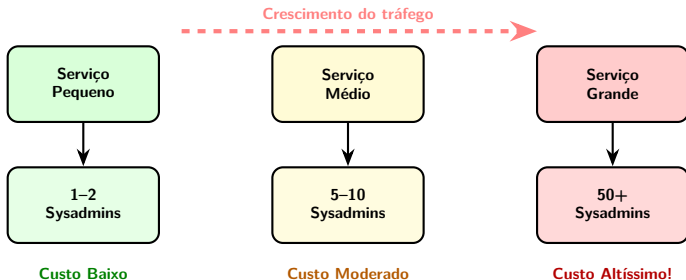
Estudo de Caso: Fábrica de Software (WebDev Studio)

Ao criar sites e portais corporativos para clientes, a *WebDev Studio* apenas clona modelos de máquinas virtuais (templates) pré-configurados com LAMP (Linux, Apache, MySQL, PHP), garantindo implantações rápidas e previsíveis sem custos de P&D em infraestrutura.

Templates Prontos + Scripts de Mercado = Implantação Sem Custo de P&D

Definição

O custo de operar um serviço com **intervenção manual** escala com o tráfego e a complexidade do sistema.



Desvantagens: Custos Indiretos

A raiz do problema

Dev e Ops possuem **experiências**, **habilidades**, **incentivos** e **vocabulários** diferentes, levando a divergências profundas.

Desenvolvimento

Lançar rápido

Inovar

Aceitar riscos

Vocabulário dev

Incentivos prod.

Operações

Estabilidade

Não mudar

Evitar riscos

Vocabulário ops

Incentivos estab.

Resultado: Patologia organizacional



Causa: A maioria das interrupções é causada por **mudanças** (nova configuração, nova funcionalidade, novo tipo de tráfego)

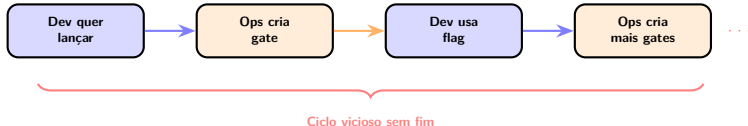
A “Guerra de Trincheiras”

Ops se defende

- Introduz **pontos de verificação** (gates)
- Revisões de lançamento com **listas extensas** de problemas passados
- Processos burocráticos cada vez mais rígidos

Dev contorna

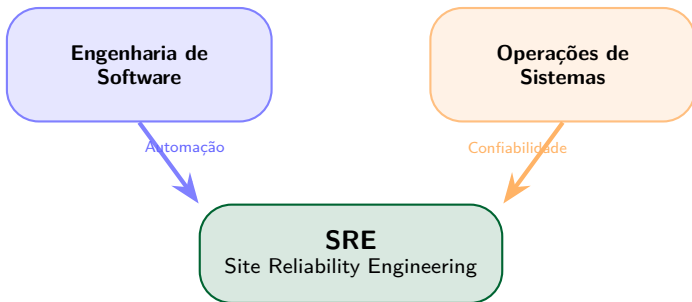
- Usa **flags de ativação** em vez de lançamentos
- Faz **atualizações incrementais**
- Divide o produto para **evitar revisão completa**
- Usa *cherry-picks*



O que é Site Reliability Engineering (SRE)?

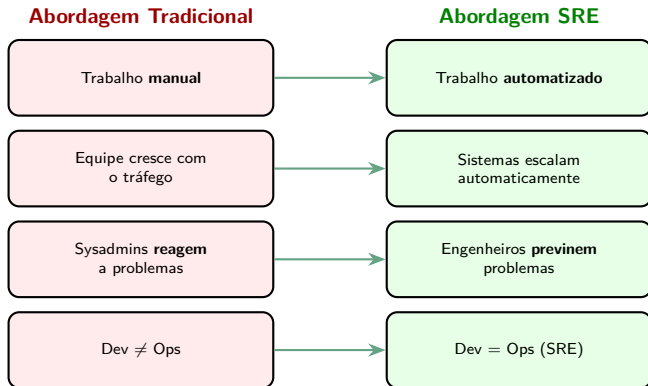
Definição (Ben Treynor Sloss, VP Google)

SRE é o que acontece quando você pede a um engenheiro de software para projetar uma equipe de operações.

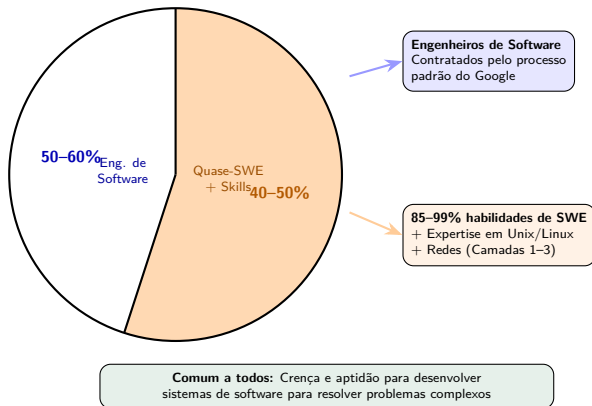


Princípio Fundamental

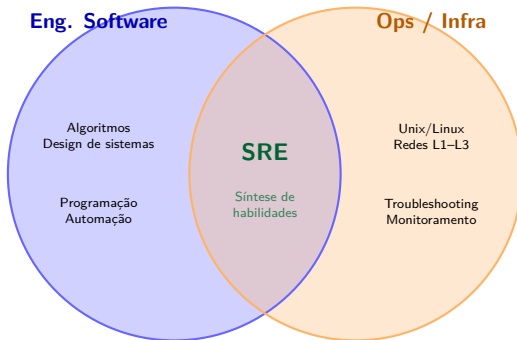
Em vez de contratar sysadmins, o Google contrata **engenheiros de software** para operar produtos e **criar sistemas automatizados** para substituir o trabalho manual.



Composição da Equipe SRE no Google



Habilidades Complementares na SRE

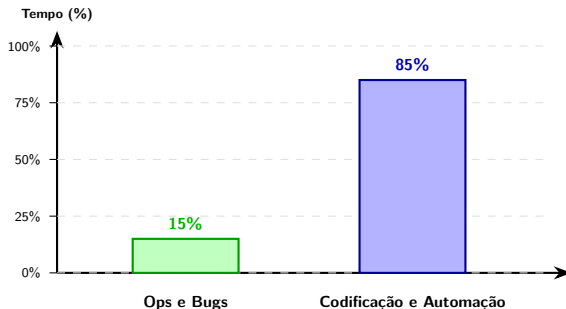


Resultado: Sistemas de alta qualidade e inteligentes —
produto da síntese de múltiplos conjuntos de habilidades

Atuação SRE: Cenário 1 — Sistema Maduro

Baixa Carga Operacional (< 20% Ops)

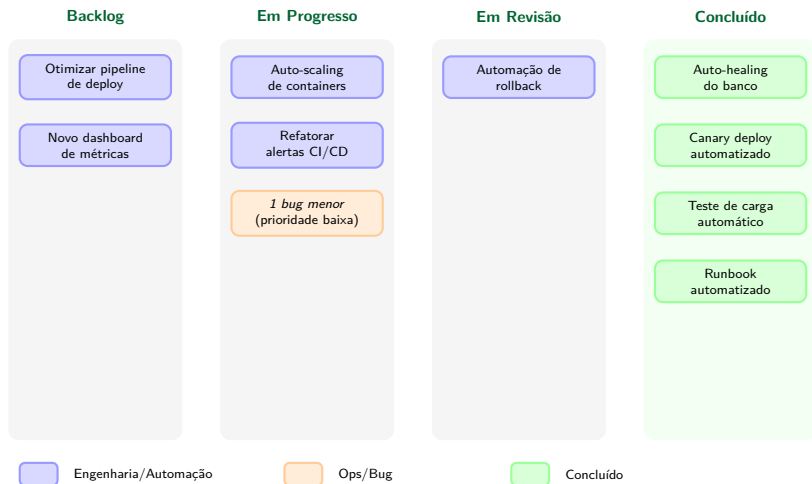
Quando o serviço é altamente automatizado, o trabalho de Ops cai drasticamente, liberando a equipe para engenharia avançada.



Conceito Google

O sistema opera e se corrige sozinho (**sistema automático**). Os SREs focam em inovação e novas funcionalidades.

Kanban do SRE: Cenário 1 — Sistema Maduro

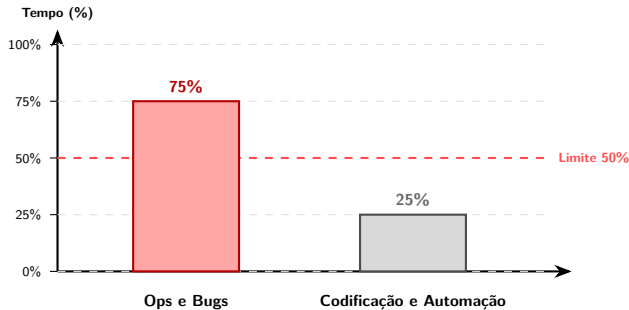


Observação

O board mostra **poucos itens de ops** e muitas **automações concluídas** — o sistema “opera sozinho”.

Alta Carga Operacional (> 50% Ops — Estado Crítico)

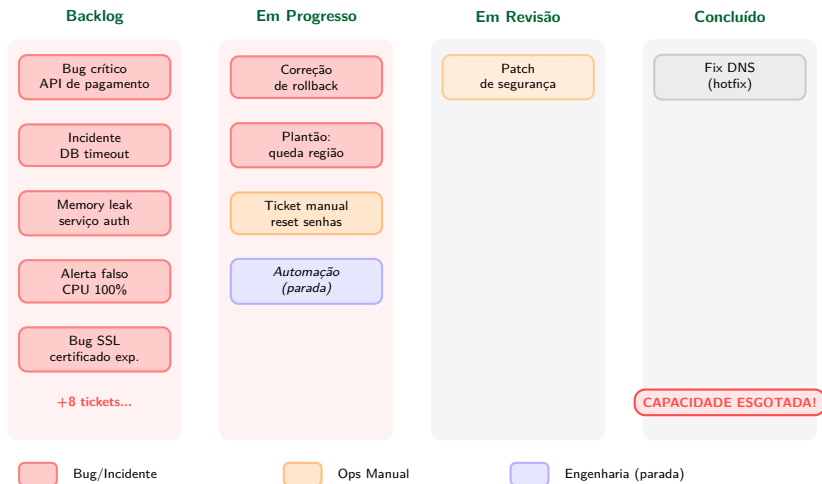
Quando ocorrem muitos bugs e chamadas de plantão, a capacidade de codificar automações é sufocada.



Ação Necessária

A equipe deve acionar a **válvula de segurança**: transferir o excesso de bugs e plantão de volta para os desenvolvedores.

Kanban do SRE: Cenário 2 — Sobrecarga de Ops



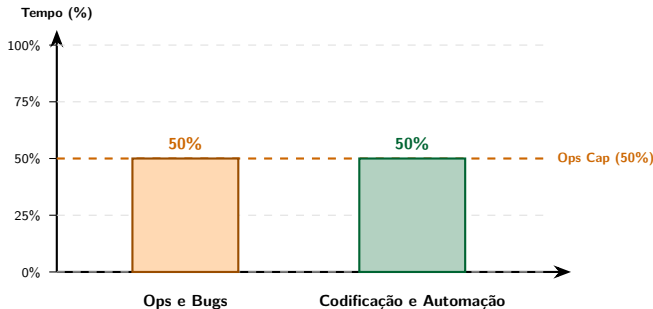
Diagnóstico

O board está **dominado por vermelho**: bugs e incidentes sufocam qualquer trabalho de engenharia. A automação está **parada**.

Atuação SRE: Cenário 3 — Equilíbrio Ideal

Distribuição Alvo do Google (Regra dos 50%)

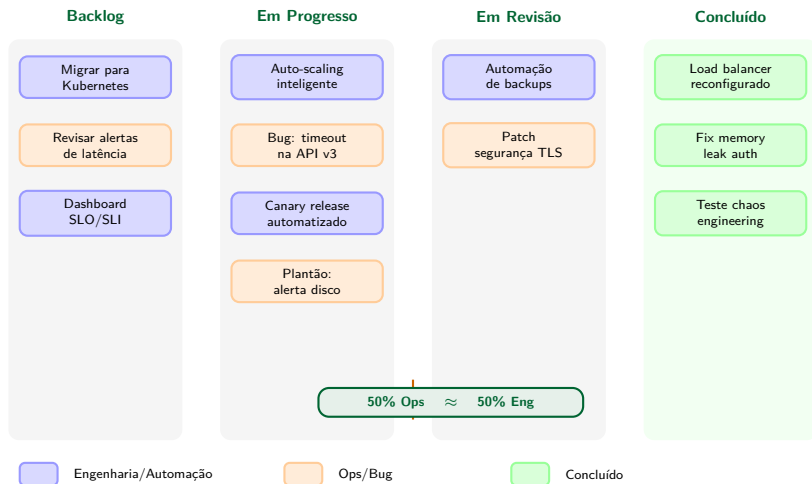
Limite superior sustentável: no máximo 50% de Ops e no mínimo 50% de Engenharia.



Impacto Sustentável

Garante que a sabedoria operacional seja mantida, enquanto reserva tempo para **engenharia autônoma e criativa**.

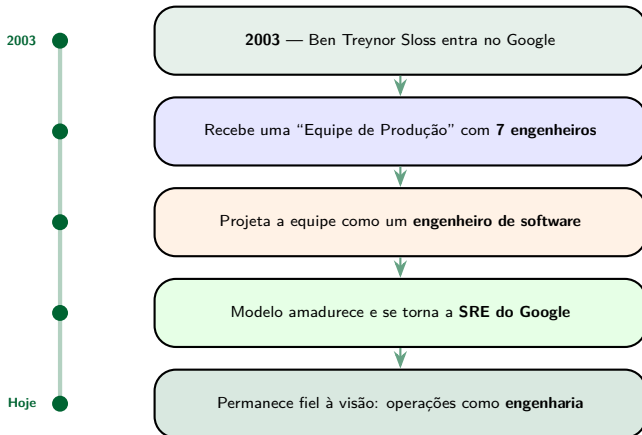
Kanban do SRE: Cenário 3 — Equilíbrio Ideal



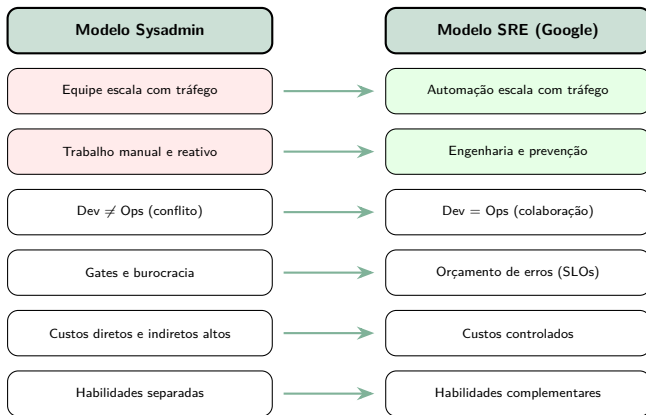
Equilíbrio Sustentável

O board mostra **cards azuis e laranjas em proporção similar**: ops sob controle e engenharia avançando continuamente.

A Origem da SRE no Google

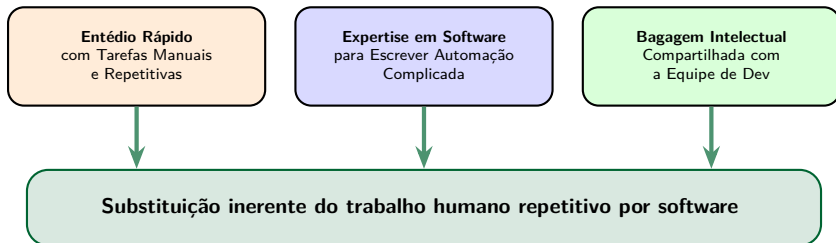


Resumo: Sysadmin vs SRE



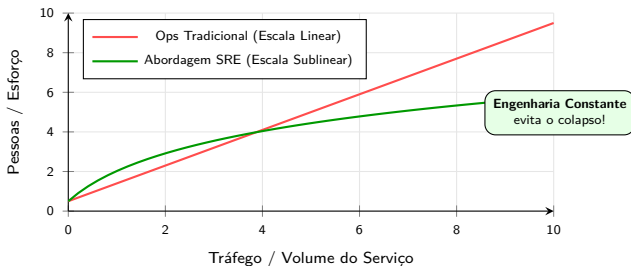
O Perfil Comportamental do SRE

O processo seletivo da SRE busca profissionais com características muito específicas:



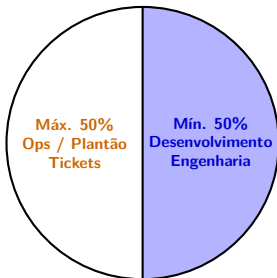
O Risco da Escala Tradicional

Se o serviço cresce e a carga de operações for tratada manualmente, a equipe precisará crescer **linearesmente com o tráfego**.



O Limite de Trabalho Operacional (Ops Cap)

O Google estabelece um **limite estrito de no máximo 50%** de tempo dedicado a tarefas operacionais agregadas (tickets, plantão/on-call, tarefas manuais).



Objetivo do Limite:

Garantir tempo de cronograma para estabilizar e melhorar o serviço.

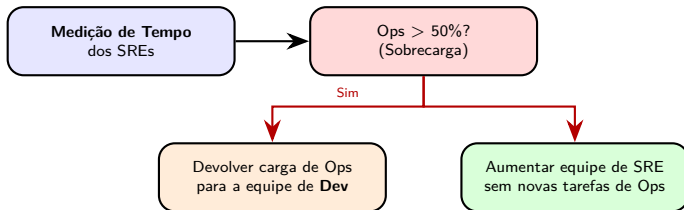
Meta de Longo Prazo:

Sistemas **automáticos** (operam e se corrigem sozinhos).

Mecanismo de Controle do Limite de 50%

Como o Google Garante o Cumprimento do Limite?

É necessário **mensurar continuamente** onde o tempo dos SREs é gasto.



Resultado: Preserva a engenharia criativa e autônoma sem perder a sabedoria de operações.

1. Inovação Rápida e Alta Aceitação de Mudança

Como SREs alteram o código diretamente, aceleram a evolução do sistema.

2. Escalabilidade com Custo Relativamente Baixo

A quantidade de SREs necessários escala de forma sublinear em relação ao sistema.

3. Treinamento Cruzado e Mobilidade Dev ↔ SRE

Transferências fáceis capacitam devs a criar sistemas de **1 milhão de núcleos de CPU**.

4. Eliminação da Disfuncionalidade Dev/Ops

Fim da guerra de trincheiras e alinhamento real de incentivos e objetivos.

Desafios Críticos Enfrentados pelo Google

Apesar dos ganhos líquidos, implementar SRE impõe barreiras organizacionais e de recrutamento.

Escassez de Talentos

Concorre com vagas de Dev;

exige alto nível em SWE
+ Engenharia de Sistemas.

Falta de Padrões

Área nova e única;

poucas referências
de mercado disponíveis.

Apoio Executivo

Decisões não ortodoxas

(ex: congelar releases
via *Error Budget*) exigem
respaldo da gerência.

Resumo da Filosofia SRE

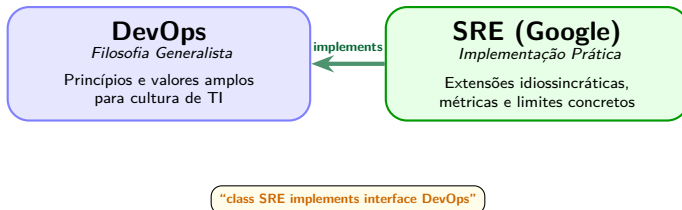
SRE não é apenas uma mudança de título, mas uma transformação cultural e de engenharia.

- **Engenharia sobre Operação Manual:** Software substitui trabalho humano.
- **Proteção de Tempo (Ops Cap 50%):** Autonomia e tempo para automação criativa.
- **Crescimento Sustentável:** Capacidade de gerenciar escala global sem explodir custos.
- **Parceria Dev e SRE:** Responsabilidade compartilhada pela confiabilidade e velocidade.

DevOps vs SRE: Qual a Relação?

A Filosofia Comum

Ambos surgiram para resolver o conflito Dev vs Ops através de automação, engenharia aplicada a operações e envolvimento em todo o ciclo de vida.



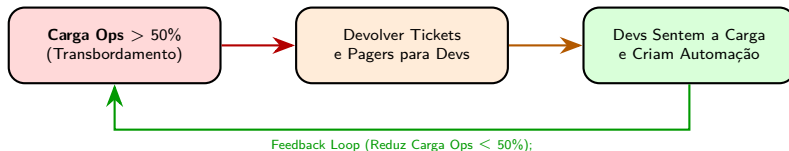
Escopo de Responsabilidade da SRE



Válvula de Segurança: Feedback para Dev

O Mecanismo de Redirecionamento

Quando o trabalho operacional estoura o limite de 50%, o excesso é devolvido para a equipe de desenvolvimento.



Alinhamento Cultural

Toda a empresa deve entender que a válvula existe para evitar que os produtos gerem carga operacional insustentável.

Regras do Plantão: Carga de Pager Ideal

Princípio Fundamental do Google SRE

Cada incidente exige aproximadamente **6 horas de trabalho dedicado** (mitigação + investigação + postmortem + ação preventiva). Por isso, um plantão de 12h suporta **no máximo 2 incidentes**.

< 1 Evento / Turno

Sub-utilização / Atrofia

- Perda de memória muscular
- Equipe desconfia dos alertas
- Falta de prática em produção

Máx. 2 Eventos / Turno

Zona Ideal de Operação

- 3h Mitigação e resposta
- 3h Causa-raiz e Postmortem
- Tempo p/ melhoria contínua

> 2 Eventos / Turno

Sobrecarga e Burnout

- Respostas superficiais (band-aid)
- Acúmulo de débito técnico
- Falta de tempo p/ postmortem

Distribuição de Tempo em um Turno de 12 Horas (Cenário Ideal):

Incidente 1 (6h: Mitigação + Postmortem) Incidente 2 (6h: Mitigação + Postmortem) Tempo Restante: Automação / Projetos

Regra de Ouro da SRE

Se o volume de alertas exceder 2 incidentes por turno com frequência, o serviço **não está pronto para a SRE** e deve ser devolvido aos Desenvolvedores para estabilização.

O que é um Postmortem?

Definição

Um **postmortem** é um **registro escrito** de um incidente, contendo: seu **impacto**, as **ações tomadas**, a(s) **causa(s) raiz** e os **itens de acompanhamento**.

Postmortem — Incidente #1042

Relatório Postmortem — Queda do Banco de Dados

Resumo do Incidente

No dia 12/03, às 14h32, o servidor de banco de dados principal ficou indisponível por 47 minutos, afetando 12.000 usuários do sistema de vendas.

Impacto

- 47 min de indisponibilidade total
- R\$ 85.000 em vendas perdidas
- 342 tickets de suporte

Cronologia

14h32 — Alerta de CPU em 100% no DB primário
14h38 — SRE de plantão inicia investigação
14h45 — Identificada query sem índice consumindo recursos
15h02 — Índice criado, query otimizada
15h19 — Sistema restaurado e estável

Causa Raiz

Deploy v2.14 introduziu uma nova consulta SQL sem índice apropriado. O aumento de tráfego às 14h30 fez o DB saturar a CPU.

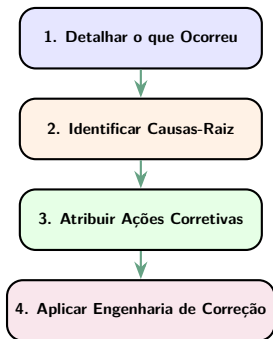
Ações Corretivas

- Adicionar revisão obrigatória de queries no CI/CD
- Criar alerta de queries lentas > 2s
- Implementar read-replica para relatórios

Lições Aprendidas

O que é um Postmortem Sem Acusações?

Documento escrito para **todos os incidentes significativos** para entender falhas sem culpar indivíduos.



Incidentes Sem Pager:
São os mais valiosos!
Revelam lacunas graves
no monitoramento.

Por que 100% de Disponibilidade é a Meta Incorreta?

O Mito dos 100%

Nenhum usuário percebe a diferença entre **99,99%** e **100%** de disponibilidade do seu serviço!



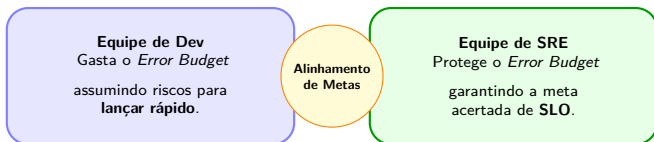
Conclusão

O esforço descomunal para buscar o último 0,001% é **perdido no ruído** das falhas de rede e dispositivos do próprio usuário.

Fórmula Fundamental do Error Budget

$\text{Error Budget} = 100\% - \text{SLO de Disponibilidade}$

Se o SLO é **99,9%**, o Error Budget é **0,1%** de indisponibilidade permitida.



Gastando o Error Budget com Segurança

Rollouts em Fases
(Canary Releases)

Experimentos de 1%
de Tráfego

Testes de Carga
em Produção

Lançamento Rápido
de Features

Interrupções dentro do Error Budget são partes normais e esperadas da inovação!

Mudança de Paradigma

A meta não é "zero interrupções", mas sim **maximizar a velocidade de lançamento** respeitando o limite do *Error Budget*.



Betsy Beyer, Chris Jones, Jennifer Petoff, Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.



Ben Treynor Sloss. “SRE é o que acontece quando você pede a um engenheiro de software para projetar uma equipe de operações.” Google, 2003–presente.